

Laboratorul 5 – Amortized Analysis

Exercitiul 1. Analiza amortizata a unui tabel dinamic

Consideram un tabel dinamic implementat printr-un vector redimensionabil. Notam:

- num – numarul de elemente stocate in tabel;
- $size$ – capacitatea curenta a tabelului.

Politica de redimensionare este urmatoarea:

- la operatia INSERT, daca $num = size$, tabelul se redimensioneaza la capacitatea $2 \cdot size$, apoi se copiaza toate elementele;
- la operatia DELETE, daca dupa stergere $num \leq \frac{size}{4}$, tabelul se redimensioneaza la capacitatea $\frac{size}{2}$.

Presupunem ca:

- inserarea sau stergerea propriu-zisa a unui element are cost 1;
- copierea unui element are cost 1;
- copierea a k elemente are cost $\Theta(k)$.

Dorim sa aratam ca pentru orice secventa de n operatii INSERT si DELETE, costul total este $O(n)$, deci costul amortizat pe operatie este $O(1)$.

1. Metoda agregata

Prin metoda agregata analizam direct costul total al unei secvente de n operatii. Vom separa costul operatiilor simple de costul redimensionarilor.

1.1. Costul operatiilor fara redimensionare

Fiecare operatie INSERT sau DELETE care nu produce redimensionare are cost constant, egal cu 1. Prin urmare, daca ignoram momentan costurile copierilor, contributia celor n operatii este cel mult:

$$O(n).$$

Ramane de analizat costul total al tuturor expansiunilor si contractiilor.

1.2. Costul total al expansiunilor

Presupunem ca la un anumit moment tabelul are capacitatea m si este plin:

$$num = size = m.$$

Atunci o operatie INSERT produce o expansiune la capacitatea $2m$. Pentru aceasta expansiune se copiaza cele m elemente existente, deci costul copierii este:

$$\Theta(m).$$

Trebuie sa aratam ca o astfel de operatie costisitoare nu poate aparea prea des.
Dupa expansiune si inserarea noului element, avem:

$$num = m + 1, \quad size = 2m.$$

Pentru ca o noua expansiune sa aiba loc, noul tabel trebuie sa devina iar plin, adica trebuie sa ajungem la:

$$num = 2m.$$

Numarul minim de inserari necesare intre aceste doua expansiuni este:

$$2m - (m + 1) = m - 1 = \Theta(m).$$

Deci o expansiune cu cost $\Theta(m)$ este separata de urmatoarea expansiune prin cel putin $\Theta(m)$ operatii ieftine.

Mai mult, daca urmarim capacitatile succesive, ele sunt:

$$1, 2, 4, 8, \dots$$

iar costurile copierilor la expansiune sunt:

$$1, 2, 4, 8, \dots$$

Daca in cadrul secventei se ajunge cel mult la o capacitate de ordin n , atunci costul total al tuturor expansiunilor este o suma geometrica:

$$1 + 2 + 4 + \dots + 2^{k-1}.$$

Aceasta suma este:

$$1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1 < 2^k.$$

Cum ultima capacitate folosita este cel mult proportionala cu n , rezulta ca:

$$1 + 2 + 4 + \dots + 2^{k-1} = O(n).$$

Prin urmare, costul total al tuturor expansiunilor este $O(n)$.

1.3. Costul total al contractiilor

Presupunem acum ca tabelul are capacitatea m si, dupa o stergere, ajungem la:

$$num \leq \frac{m}{4}.$$

Atunci tabela se contracta la capacitatea:

$$\frac{m}{2}.$$

Numarul de elemente care trebuie copiate dupa contractie este exact numarul de elemente ramase, adica cel mult:

$$\frac{m}{4}.$$

Deci costul unei contractii este:

$$\Theta\left(\frac{m}{4}\right) = \Theta(m).$$

Si aici trebuie sa aratam ca doua contractii consecutive nu pot aparea foarte aproape una de alta.

Imediat dupa contractie avem:

$$size = \frac{m}{2}, \quad num \leq \frac{m}{4}.$$

Observam ca:

$$\frac{m}{4} = \frac{1}{2} \cdot \frac{m}{2},$$

deci imediat dupa contractie tabela este cel mult pe jumatate plina.

Pentru ca o noua contractie sa aiba loc, in noul tabel trebuie sa ajungem la:

$$num \leq \frac{size}{4} = \frac{m/2}{4} = \frac{m}{8}.$$

Prin urmare, intre doua contractii consecutive trebuie sa se mai stearga cel putin:

$$\frac{m}{4} - \frac{m}{8} = \frac{m}{8} = \Theta(m)$$

elemente.

Deci si o contractie cu cost $\Theta(m)$ este separata de urmatoarea contractie prin cel putin $\Theta(m)$ operatii ieftine.

Rezulta ca suma costurilor tuturor contractiilor dintr-o secventa de n operatii este tot:

$$O(n).$$

1.4. Concluzie pentru metoda agregata

Adunand cele trei contributii:

- costul operatiilor simple: $O(n)$;
- costul total al expansiunilor: $O(n)$;
- costul total al contractiilor: $O(n)$;

obtinem:

$$T(n) = O(n).$$

Prin urmare, costul amortizat pe operatie este:

$$\frac{T(n)}{n} = O(1).$$

2. Metoda contabila

Prin metoda contabila atribuim fiecarei operatii un cost amortizat mai mare decat costul ei real, iar diferenta este stocata ca *credit*. Acest credit va fi folosit ulterior pentru a plati operatiile scumpe de redimensionare. Conditia esentiala este ca totalul creditelor stocate sa nu devina niciodata negativ.

Alegem urmatoarele costuri amortizate:

- fiecarui INSERT ii atribuim cost amortizat 3;
- fiecarui DELETE ii atribuim cost amortizat 2.

2.1. Invariantul de credit

Folosim urmatorul invariant:

- fiecare element aflat in jumatatea dreapta a tabelului are asociate 2 credite;
- fiecare pozitie goala aflata in jumatatea stanga a tabelului are asociat 1 credit.

Trebuie sa aratam ca acest invariant este suficient pentru a plati toate expansiunile si toate contractiile.

2.2. Operatia INSERT fara redimensionare

Costul real este:

$$c_i = 1.$$

Costul amortizat perceput este:

$$\hat{c}_i = 3.$$

Din cele 3 unitati:

- 1 unitate plateste inserarea propriu-zisa;
- celelalte 2 unitati sunt pastrate ca credit, astfel incat invariantul sa ramana adevarat.

Asadar, fiecare inserare simpla produce credit suplimentar care poate fi folosit mai tarziu.

2.3. Operatia INSERT cu expansiune

Presupunem ca inainte de operatie tabelul este plin:

$$num = size = m.$$

La expansiune se copiaza cele m elemente existente, apoi se insereaza elementul nou. Costul real este:

$$c_i = m + 1.$$

Sa vedem de unde vine plata pentru copierea celor m elemente.

Cand tabela este plina, jumatatea dreapta contine exact:

$$\frac{m}{2}$$

elemente. Fiecare dintre aceste elemente are cate 2 credite, conform invariantului. Prin urmare, totalul creditelor disponibile pe jumatatea dreapta este:

$$2 \cdot \frac{m}{2} = m.$$

Aceste m credite acopera exact costul copierii celor m elemente in noua tabela.

Dupa dublarea capacitatii, vechile elemente ajung in jumatatea stanga a noii tabele, deci nu mai este necesar sa pastreze acele credite. Inserarile ulterioare vor reconstrui treptat creditul necesar pentru o viitoare expansiune.

2.4. Operatia DELETE fara redimensionare

Costul real este:

$$c_i = 1.$$

Costul amortizat perceput este:

$$\hat{c}_i = 2.$$

Din cele 2 unitati:

- 1 unitate plateste stergerea;
- 1 unitate ramane drept credit pe o pozitie goala din jumatatea stanga.

Aceste credite vor fi folosite pentru a plati o contractie viitoare.

2.5. Operatia DELETE cu contractie

Presupunem ca inainte de contractie capacitatea este m , iar dupa stergere se ajunge la:

$$num \leq \frac{m}{4}.$$

Atunci capacitatea devine:

$$\frac{m}{2}.$$

Costul copierii este proportional cu numarul de elemente ramase, adica cel mult:

$$\frac{m}{4}.$$

Deci costul real este:

$$c_i \leq 1 + \frac{m}{4}.$$

Trebuie sa aratam ca exista suficiente credite pentru a plati aceasta copiere.

Imediat inainte de contractie, in partea stanga a tabelului exista suficiente pozitii goale care au acumulat credite prin stingerile anterioare. Mai precis, pentru a cobori de la o stare in care tabela era pe jumatate plina pana la pragul de un sfert, au fost necesare cel putin:

$$\frac{m}{2} - \frac{m}{4} = \frac{m}{4}$$

stingeri, iar fiecare stingere a depus cate un credit. Aceste cel putin $\frac{m}{4}$ credite sunt suficiente pentru a plati copierea celor cel mult $\frac{m}{4}$ elemente ramase.

2.6. Concluzie pentru metoda contabilă

Invariantul de credit ramane adevarat dupa fiecare operatie, deci totalul creditelor stocate nu devine niciodata negativ. In consecinta,

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i.$$

Cum fiecare operatie are cost amortizat constant, rezulta ca:

$$\sum_{i=1}^n \hat{c}_i = O(n).$$

Prin urmare,

$$\sum_{i=1}^n c_i = O(n),$$

iar costul amortizat pe operatie este:

$$\frac{1}{n} \sum_{i=1}^n c_i = O(1).$$

3. Metoda potentialului

Pentru metoda potentialului, introducem factorul de incarcare:

$$\alpha(D) = \frac{num}{size}.$$

Ideea este sa definim o functie potential care:

- sa fie 0 atunci cand tabela este “echilibrata”, adica la jumatate plina;
- sa acumuleze suficienta “energie” atunci cand tabela se apropie de plin, pentru a plati expansiunea;
- sa acumuleze suficienta “energie” atunci cand tabela se apropie de un sfert plin, pentru a plati contractia.

Definim:

$$\Phi(D) = \begin{cases} 2 \textit{num} - \textit{size}, & \text{daca } \alpha(D) \geq \frac{1}{2}, \\ \frac{\textit{size}}{2} - \textit{num}, & \text{daca } \alpha(D) < \frac{1}{2}. \end{cases}$$

3. Metoda potentialului

Definim factorul de incarcare:

$$\alpha(D) = \frac{\textit{num}}{\textit{size}}.$$

Alegem functia potential:

$$\Phi(D) = \begin{cases} 2 \textit{num} - \textit{size}, & \text{daca } \alpha(D) \geq \frac{1}{2}, \\ \frac{\textit{size}}{2} - \textit{num}, & \text{daca } \alpha(D) < \frac{1}{2}. \end{cases}$$

Aceasta alegere este naturala deoarece:

- la $\alpha = \frac{1}{2}$ avem $\Phi(D) = 0$, deci tabela este intr-o stare de echilibru;
- cand tabela se apropie de plin, potentialul creste si poate plati o expansiune;
- cand tabela se apropie de pragul $\alpha = \frac{1}{4}$, potentialul creste din nou si poate plati o contractie.

Mai mult, functia este nenegativa:

$$\alpha(D) \geq \frac{1}{2} \Rightarrow 2 \textit{num} - \textit{size} \geq 0, \quad \alpha(D) < \frac{1}{2} \Rightarrow \frac{\textit{size}}{2} - \textit{num} > 0.$$

Costul amortizat al operatiei i este:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Vom analiza cele patru tipuri de operatii.

Cazul 1: INSERT fara expansiune

Costul real este:

$$c_i = 1.$$

Daca nu apare redimensionare, atunci *size* ramane constant si *num* creste cu 1. In ambele ramuri ale functiei potential, variatia lui Φ este marginita de o constanta:

$$\Phi(D_i) - \Phi(D_{i-1}) \in \{-1, 0, 2\}.$$

Prin urmare:

$$\hat{c}_i = 1 + O(1) = O(1).$$

Cazul 2: INSERT cu expansiune

Presupunem ca inainte de operatie:

$$num = size = m.$$

Atunci costul real este:

$$c_i = m + 1,$$

deoarece copiem cele m elemente existente si apoi inseram elementul nou.

Inainte de operatie:

$$\Phi(D_{i-1}) = 2m - m = m.$$

Dupa expansiune:

$$size' = 2m, \quad num' = m + 1,$$

deci:

$$\Phi(D_i) = 2(m + 1) - 2m = 2.$$

Rezulta:

$$\hat{c}_i = (m + 1) + (2 - m) = 3 = O(1).$$

Cazul 3: DELETE fara contractie

Costul real este:

$$c_i = 1.$$

Daca nu apare contractie, atunci *size* ramane constant si *num* scade cu 1. In ambele ramuri, variatia potentialului este din nou marginita de o constanta:

$$\Phi(D_i) - \Phi(D_{i-1}) \in \{-2, 1, 2\}.$$

Prin urmare:

$$\hat{c}_i = 1 + O(1) = O(1).$$

Cazul 4: DELETE cu contractie

Presupunem ca inainte de operatie:

$$size = m, \quad num = \frac{m}{4} + 1.$$

Dupa stergere:

$$num' = \frac{m}{4},$$

deci are loc contractia la:

$$size' = \frac{m}{2}.$$

Costul real este:

$$c_i = 1 + \frac{m}{4},$$

deoarece stergem un element si copiem cele $\frac{m}{4}$ elemente ramase.

Inainte de operatie:

$$\Phi(D_{i-1}) = \frac{m}{2} - \left(\frac{m}{4} + 1\right) = \frac{m}{4} - 1.$$

Dupa contractie:

$$\alpha(D_i) = \frac{m/4}{m/2} = \frac{1}{2},$$

deci:

$$\Phi(D_i) = 0.$$

Prin urmare:

$$\hat{c}_i = \left(1 + \frac{m}{4}\right) + \left(0 - \left(\frac{m}{4} - 1\right)\right) = 2 = O(1).$$

Concluzie

In toate cazurile, costul amortizat al unei operatii este constant:

$$\hat{c}_i = O(1).$$

Deci pentru orice secventa de n operatii:

$$\sum_{i=1}^n \hat{c}_i = O(n).$$

Cum $\Phi(D_0) = 0$ si $\Phi(D_i) \geq 0$ pentru orice i , rezulta:

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i = O(n).$$

Prin urmare:

$$T(n) = O(n),$$

iar costul amortizat pe operatie este:

$$O(1).$$

Concluzia finala pentru Exerciitiul 1

Prin toate cele trei metode – agregata, contabila si potentialului – am demonstrat acelasi rezultat: $T(n)=O(n)$, pentru orice secventa de n operatii INSERT si DELETE. Deci costul amortizat pe operatie este: $O(1)$.

Exerciitiul 2. Structura cu mai multe tablouri sortate

Dorim sa memoram o multime de n elemente folosind mai multe tablouri sortate, in locul unui singur tablou sortat.

Fie

$$k = \lceil \log_2(n + 1) \rceil.$$

Structura este formata din tablourile

$$A_0, A_1, \dots, A_{k-1},$$

unde tabloul A_i are capacitatea

$$2^i.$$

Fiecare tablou este fie *gol*, fie *complet plin*. Daca scriem pe n in baza 2 sub forma

$$\langle n_{k-1}, n_{k-2}, \dots, n_0 \rangle,$$

atunci tabloul A_i este plin daca si numai daca

$$n_i = 1.$$

Aceasta corespondenta este naturala deoarece fiecare tablou A_i , atunci cand este plin, contine exact 2^i elemente. Prin urmare, numarul total de elemente stocate in structura este

$$\sum_{i=0}^{k-1} n_i 2^i = n.$$

Fiecare tablou este sortat intern, dar intre doua tablouri diferite nu exista nicio relatie de ordine. Cu alte cuvinte, este posibil ca un element din A_i sa fie mai mare sau mai mic decat un element din A_j , fara nicio regula globala.

1. Operatia SEARCH

1.1. Descrierea ideii

Deoarece fiecare tablou plin este sortat, putem cauta un element x intr-un tablou plin folosind cautare binara.

Cum nu exista ordine intre tablouri diferite, nu putem sti dinainte in care tablou se afla x . De aceea, in cel mai general caz, trebuie sa verificam toate tablourile pline:

- pentru fiecare tablou A_i care este plin, executam $\text{BINARYSEARCH}(A_i, x)$;

- daca elementul este gasit intr-unul dintre tablouri, intoarcem **true**;
- daca nu este gasit in niciun tablou plin, intoarcem **false**.

Aceasta procedura este corecta deoarece orice element memorat in structura apartine exact unuia dintre tablourile pline.

1.2. Pseudocod complet pentru SEARCH

Algorithm 1 SEARCH(x)

```

1: for  $i \leftarrow 0$  to  $k - 1$  do
2:   if  $A_i$  nu este gol then
3:      $poz \leftarrow \text{BINARYSEARCH}(A_i, x)$ 
4:     if  $poz \neq \text{NIL}$  then
5:       return TRUE
6:     end if
7:   end if
8: end for
9: return FALSE

```

Aici:

- BINARYSEARCH(A_i, x) intoarce pozitia lui x in tabloul A_i daca elementul exista;
- daca elementul nu exista in A_i , functia intoarce NIL.

1.3. De ce pseudocodul este corect

Pseudocodul parcurge toate tablourile care pot contine elemente, adica toate tablourile pline. Intrucat elementele sunt memorate numai in aceste tablouri, daca x exista in structura, atunci el trebuie sa se afle intr-un anumit tablou plin A_i . In momentul in care algoritmul ajunge la acel tablou, cautarea binara il va gasi, iar algoritmul intoarce **true**.

Daca algoritmul termina parcurgerea fara sa gaseasca elementul, atunci x nu se afla in niciun tablou plin si, deci, nu apartine multimii memorate. In acest caz, intoarcerea valorii **false** este corecta.

1.4. Analiza worst-case a operatiei SEARCH

Tabloul A_i are dimensiunea

$$|A_i| = 2^i.$$

Cautarea binara intr-un tablou de dimensiune 2^i are cost

$$O(\log 2^i).$$

Cum

$$\log_2(2^i) = i,$$

rezulta ca timpul necesar pentru a cauta in tabloul A_i este

$$O(i).$$

In cel mai nefavorabil caz, toate tablourile

$$A_0, A_1, \dots, A_{k-1}$$

sunt pline si elementul cautat nu este gasit decat dupa verificarea tuturor, sau nu exista deloc. Atunci costul total este

$$\sum_{i=0}^{k-1} O(i) = O\left(\sum_{i=0}^{k-1} i\right).$$

Stim ca

$$\sum_{i=0}^{k-1} i = \frac{k(k-1)}{2},$$

deci

$$\sum_{i=0}^{k-1} i = O(k^2).$$

Cum

$$k = \lceil \log_2(n+1) \rceil,$$

obtinem

$$k = O(\log n),$$

de unde rezulta

$$O(k^2) = O((\log n)^2).$$

Concluzie. Complexitatea in worst-case a operatiei SEARCH este

$$O((\log n)^2).$$

2. Operatia INSERT

2.1. Descrierea ideii

Pentru a insera un element nou x , pornim de la un tablou sortat de dimensiune 1 care contine doar elementul x .

Apoi incercam sa-l plasam la nivelul 0:

- daca A_0 este gol, inserarea se termina;
- daca A_0 este plin, nu putem pastra doua tablouri diferite de dimensiune 1, deci le interclasam si obtinem un tablou sortat de dimensiune 2.

Continuam acelasi proces:

- daca A_1 este gol, plasam acolo noul tablou;
- daca A_1 este plin, interclasam din nou si obtinem un tablou de dimensiune 4;
- continuam pana la primul nivel liber.

Aceasta procedura este analoga incrementarii unui contor binar:

- un tablou plin corespunde unui bit 1;
- un tablou gol corespunde unui bit 0;
- interclasarea cu propagare la nivelul urmator corespunde propagarii transportului (*carry*) in adunarea binara.

2.2. Pseudocod complet pentru MERGE

Pentru ca pseudocodul operatiei INSERT sa fie complet, definim mai intai explicit operatia de interclasare a doua tablouri sortate.

Algorithm 2 MERGE(X, Y)

```

1: Input: doua tablouri sortate  $X$  si  $Y$ , fiecare de dimensiune  $m$ 
2: Output: un tablou sortat  $Z$  de dimensiune  $2m$  ce contine toate elementele din  $X$  si  $Y$ 
3: aloca tabloul  $Z[1 \dots 2m]$ 
4:  $i \leftarrow 1, j \leftarrow 1, t \leftarrow 1$ 
5: while  $i \leq m$  and  $j \leq m$  do
6:   if  $X[i] \leq Y[j]$  then
7:      $Z[t] \leftarrow X[i]$ 
8:      $i \leftarrow i + 1$ 
9:   else
10:     $Z[t] \leftarrow Y[j]$ 
11:     $j \leftarrow j + 1$ 
12:   end if
13:    $t \leftarrow t + 1$ 
14: end while
15: while  $i \leq m$  do
16:    $Z[t] \leftarrow X[i]$ 
17:    $i \leftarrow i + 1$ 
18:    $t \leftarrow t + 1$ 
19: end while
20: while  $j \leq m$  do
21:    $Z[t] \leftarrow Y[j]$ 
22:    $j \leftarrow j + 1$ 
23:    $t \leftarrow t + 1$ 
24: end while
25: return  $Z$ 

```

Acest algoritm este standard si are cost liniar in numarul total de elemente procesate. Daca fiecare dintre cele doua tablouri are dimensiunea m , atunci MERGE are cost

$$\Theta(2m) = \Theta(m).$$

2.3. Pseudocod complet pentru INSERT

Algorithm 3 INSERT(x)

```

1: construiesc un tablou  $B$  de dimensiune 1 care contine doar elementul  $x$ 
2:  $i \leftarrow 0$ 
3: while  $i < k$  and  $A_i$  nu este gol do
4:    $B \leftarrow \text{MERGE}(B, A_i)$ 
5:   marcheaza  $A_i$  drept gol
6:    $i \leftarrow i + 1$ 
7: end while
8: if  $i = k$  then
9:   creeaza un nou nivel  $A_k$  si seteaza  $k \leftarrow k + 1$ 
10: end if
11:  $A_i \leftarrow B$ 
12: marcheaza  $A_i$  drept plin

```

2.4. De ce pseudocodul este corect

La fiecare pas al buclei **while**, tabloul B este sortat si are dimensiunea 2^i . Daca A_i este plin, atunci si el are exact dimensiunea 2^i si este sortat. Prin urmare, MERGE poate fi aplicat corect, iar rezultatul este un nou tablou sortat de dimensiune

$$2^i + 2^i = 2^{i+1}.$$

Dupa interclasare, nivelul i devine gol, iar noul tablou B trebuie plasat la nivelul urmator. Procedura continua pana cand gasim primul nivel liber. In acel moment, tabloul B este plasat acolo si toate proprietatile structurii raman adevarate:

- fiecare nivel este fie gol, fie complet plin;
- fiecare tablou plin este sortat;
- numarul total de elemente creste cu exact 1.

Prin urmare, pseudocodul descrie corect operatia INSERT.

3. Analiza worst-case pentru INSERT

Cel mai nefavorabil caz apare atunci cand toate tablourile

$$A_0, A_1, \dots, A_{k-1}$$

sunt pline. In acest caz, inserarea produce o cascada completa de interclasari.

La nivelul i , se interclaseaza doua tablouri de dimensiune 2^i . Costul unei astfel de interclasari este proportional cu numarul total de elemente procesate, deci

$$\Theta(2^i).$$

Prin urmare, costul total al unei singure operatii INSERT in cel mai nefavorabil caz este

$$1 + 2 + 4 + \dots + 2^{k-1}.$$

Aceasta suma este geometrica:

$$1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1.$$

Cum

$$k = \lceil \log_2(n+1) \rceil,$$

rezulta ca

$$2^k = O(n).$$

Mai precis, din definitia functiei plafon avem

$$k - 1 < \log_2(n+1) \leq k,$$

de unde

$$2^{k-1} < n+1 \leq 2^k.$$

Prin urmare, 2^k este de acelasi ordin cu n , adica

$$2^k = O(n).$$

Rezulta:

$$2^k - 1 = O(n).$$

Concluzie. Complexitatea in worst-case a operatiei INSERT este

$$O(n).$$

4. Analiza amortizata a lui INSERT prin metoda agregata

Consideram o secventa de n operatii INSERT.

Observatia esentiala este ca un element inserat poate participa la mai multe interclasari, dar de fiecare data cand participa la una, el ajunge intr-un tablou de dimensiune dubla.

Mai exact, un element poate trece prin tablouri de dimensiuni:

$$1, 2, 4, 8, \dots$$

Daca structura contine in total cel mult n elemente, atunci dimensiunea maxima relevanta a unui tablou este de ordin n . Prin urmare, numarul maxim de dublari prin care poate trece un element este cel mult

$$O(\log n).$$

Aceasta inseamna ca fiecare element poate fi mutat de cel mult $O(\log n)$ ori pe intreaga durata a secventei.

Cum in total inseram n elemente, costul total al tuturor mutarilor/interclasarilor este

$$O(n \log n).$$

Impartind la numarul total de inserari, obtinem costul amortizat per operatie:

$$\frac{O(n \log n)}{n} = O(\log n).$$

Concluzie. Prin metoda agregata, complexitatea amortizata a operatiei INSERT este

$$O(\log n).$$

5. Analiza amortizata a lui INSERT prin metoda contabila

Aplicam acum metoda contabila.

Ideea este sa incarcam fiecare operatie INSERT cu suficient credit pentru a plati nu doar inserarea curenta, ci si toate mutarile viitoare ale elementului nou introdus.

Am aratat deja ca un element poate fi mutat de cel mult

$$O(\log n)$$

ori, deoarece la fiecare mutare ajunge intr-un tablou de dimensiune dubla.

Prin urmare, daca asociem elementului nou inserat un credit total de ordin

$$O(\log n),$$

acest credit este suficient pentru toate interclasariile viitoare la care elementul va participa.

Astfel:

- o parte din costul amortizat plateste inserarea curenta;
- restul este stocat sub forma de credit si va fi consumat pe parcursul mutarilor viitoare.

Deoarece fiecare element este mutat de cel mult un numar logarithmic de ori, creditul initial este suficient. Rezulta ca totalul creditelor nu devine negativ, iar suma costurilor reale este dominata de suma costurilor amortizate:

$$\sum c_i \leq \sum \hat{c}_i.$$

Daca fiecarui INSERT ii atribuim cost amortizat

$$O(\log n),$$

atunci pentru n operatii obtinem

$$\sum \hat{c}_i = O(n \log n).$$

Prin urmare,

$$\sum c_i = O(n \log n),$$

iar costul amortizat pe operatie este

$$\frac{O(n \log n)}{n} = O(\log n).$$

Concluzie. Prin metoda contabilă, complexitatea amortizată a operației INSERT este $O(\log n)$.

Concluzia finală pentru Exercițiul 2

Pentru structura formată din tablourile sortate

$$A_0, A_1, \dots, A_{k-1},$$

obținem:

- operația SEARCH are complexitate în worst-case

$$O((\log n)^2);$$

- operația INSERT are complexitate în worst-case

$$O(n);$$

- operația INSERT are complexitate amortizată

$$O(\log n),$$

atât prin metoda agregată, cât și prin metoda contabilă.